=======

Reporte: Paradigmas de la Programación Práctica 1:

Actividad: Cola de impresión en lenguaje C

Alumno: Guzmaán Ramírez Carlos Alberto

Maestro: Jose Carlos Gallegos Mariscal

1. Objetivo de la practica:

Comprender y aplicar: entornos y reglas de alcance, manejo de memoria estática y dinámica, estructuras de control, subprogramas, y tipos de datos.

2. Especificaciones del problema:

Comportamiento esperado:

- El programa debe de poseer una cola FIFO.
- Una función peek que no modifique la cola y solo consulte al frente.
- La función dequeue si modificara la cola y la recorrera.
- Print/listar debe de mostrar los trabajos en el orden de atención

Parte 1:

El primero código creara una estructura con un tipo de dato llamado trabajo construido de la siguiente forma:

```c
typedef struct Trabajo
{
    int id;
    char usuario[50];
    char documento[50];
    int total_pags;
    int restant_pags;
    int copias;
    int priordad;
    int estado;
} Trabajo;
```

Funcionamiento de cada variable

- La ID es el identificador del trabajo
- Usuario es quien demanda la impresión.
- Documento es el nombre de la impresión.
- Total_pags Son el total de páginas a imprimir.
- Restant_pags es el total de páginas faltantes a imprimir definido por hojas * copias.

- Copias son las cantidades de veces que se imprimiran el total de páginas.
- Prioridad es el nivel de importancia que el documento tiene ante otros.
- Estado es para indicar si se esta imprimiendo o si esta en espera.

Menú de la parte 1:

1. Permite agregar una impresión al vector a través de solicitudes en una función con getch()
2. Muestra el archivo más pronto a imprimir
3. imprime el primer archivo en la lista
4. Muestra una lista completa con la información de cada archivo.

Funciones utilizadas:

```
void q_init(trabajo *lista, int total)
{
    Esta función captura la información de la impresión y la añade a la lista.
}

void q_print(trabajo *lista, int total)
{
    Imprime todos los documentos faltantes sin modificar la lista.
}

void q_dequeue(trabajo *lista, int *total)
{
    Imprime el documento más proximo modificando la lista, eliminando a este.
}
void eliminar(trabajo *lista, int indice, int *total)
{
    Es la ayuda del d_dequeue y recorre la lista para eliminar el archivo impreso.
}

void imprimir(trabajo *lista, int i)
{
    Imprime unicamente un trabajo en x indice, utilizado por la función q_print.
}

void q_peak(trabajo *lista, int total)
{
    Muestra el primer documento faltante con mayor prioridad.
}

int q_is_empty(int total)
{
    Verifica si el vector esta vacio para continuar o detenerse dependiendo del
resultado.
}

float valida_num(int min, int max, int flotante, const char *texto)
{
    Permite la captura de números ya sean enteros o flotantes.
}
```

```
char *valida_char(char *texto, const char *msg)
{
    Permite la captura de caracteres en cadenas limitando los caracteres
especiales que permite introducir.
}
```

Parte 2:

El segundo código trata de crear un código que funcione similar al primero pero con la diferencia de tener una estructura tipo lista para marcar el inicio y final de la lista, así como la utilización de memoria dinámica a través del malloc y liberando la memoria al final.

Estructura:

```
typedef struct Ts_Trabajo
{
    int id;
    char usuario[32];
    char documento[32];
    int total_pgs;
    int restante_pgs;
    int prioridad;
    int copias;
    int estado;
    // Ts_Trabajo *siguiente;
} Trabajo;

typedef struct Nodo
{
    Trabajo job;
    struct Nodo *siguiente;
} Nodo;

typedef struct Ts_Lista
{
    Nodo *cabeza;
    Nodo *cola;
    int tamano;
} Ts_Lista;
```

Funcionamiento de cada variable

- La ID es el identificador del trabajo
- Usuario es quien demanda la impresión.
- Documento es el nombre de la impresión.

- Total_pags Son el total de páginas a imprimir.
- Restant_pags es el total de páginas faltantes a imprimir definido por hojas * copias.
- Copias son las cantidades de veces que se imprimiran el total de páginas.
- Trabajo job es una variable que posee el trabajo, creada para indicar el siguiente en estos trabajos.
- Struct nodo *siguiente es una variable tipo struct con el puntero de la ubicación de la siguiente cancion.
- La siguiente estructura posee Cabeza, Cola y Tamaño lo cual sería el principio, final y el tamaño de la lista.

3. Funciones del programa

qd_init()

Inicializa la lista con cola, cabeza y tamaño en NULL / 0

qd_enqueue()

Misma función que la parte anterior con la diferencia de que recorre la lista para imprimir el primero mientras que la cabeza no sea null (lista vacia)

q_peak()

Imprime la cabeza, que es el trabajo que se encuentra más enfrente.

qd_dequeue()

Permite ingresar los trabajos a la lista dependiendo si ya hay uno antes o es el primero.

mostrarLista()

Recorre la cola completa mostrando todos los trabajos pendientes.

qd_destroy()

Libera toda la memoria utilizada por la cola antes de finalizar el programa.

Resto de funciones

Las demas funciones son exactamente iguales a las de la parte 1.

La memoria dinámica se reserva con el malloc:

```
nodo(malloc(sizeof(Nodo)))
```

Parte 3:

El programa simula el proceso de impresión página por página.

Durante la ejecución de

```
qd_dequeue()
```

se realiza lo siguiente:

- Se identifica el trabajo en la cabeza de la cola.
- Se imprime el número de páginas restantes.
- Se reduce el número de páginas faltantes.
- Se utiliza un pequeño retraso para simular el tiempo de impresión.

Ejemplo del delay, que simula una pausa de impresión (aunque sigue siendo muy rápido):

```
Sleep(50);
```

Este retraso se ejecuta dentro de un ciclo que simula la impresión de cada página.

También se muestran puntos en pantalla para representar el progreso del trabajo.

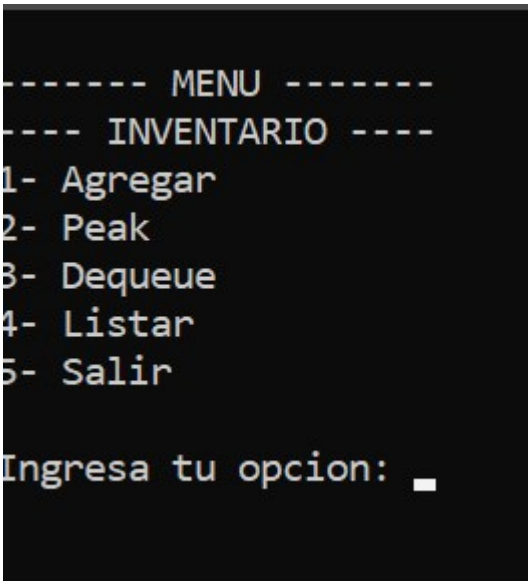
Una vez que todas las páginas han sido impresas, el trabajo se elimina de la cola y se libera su memoria.

6. Salidas

A continuacion la salida de los programas

Ejercicio 1: Lista estatica

Menu



Agregar:

```
Ingresa el usuario quien mando a imprimir: ACTIVATUMENTE
Ingresa el nombre del decumento: DEXTER
Ingresa el numero total de paginas: 20
Ingresa el numero total de copias: 2
1- Normal
2- Urgente

Ingresa la prioridad: 2
```

Peak:

```
ID  Usuario          Documento            Total_Pag   Restantes_pag   Copias  Prioridad   Estado
1   ACTIVATUMENTE    DEXTER.pdf           20          40              2       Urgente   En Cola
------- MENU -------
---- INVENTARIO ----
1- Agregar
2- Peak
3- Dequeue
4- Listar
5- Salir

Ingresa tu opcion: _
```

Dequeue:

```
Completado
------- MENU -------
---- INVENTARIO ----
1- Agregar
2- Peak
3- Dequeue
4- Listar
5- Salir

Ingresa tu opcion: _
```

Al haber imprimido la anterior no se muestra en la lista

Listar:

```
La lista esta vacia

------- MENU -------
---- INVENTARIO ----
1- Agregar
2- Peak
3- Dequeue
4- Listar
5- Salir

Ingresa tu opcion:
```

Ejercicio 3: Lista dinamica

Menu:

```
La lista esta vacia

------- MENU -------
---- INVENTARIO ----
1- Agregar
2- Peak
3- Dequeue
4- Listar
5- Salir

Ingresa tu opcion:
```

Agregar:

```
Ingresa el usuario quien mando a imprimir: BOLSA
Ingresa el nombre del decumento: TAREA
Ingresa el numero total de paginas: 5
Ingresa el numero total de copias: 2
1- Normal
2- Urgente

Ingresa la prioridad: 1
------- MENU -------
---- INVENTARIO ----
1- Agregar
2- Peak
3- Dequeue
4- Listar
5- Salir

Ingresa tu opcion: _
```

Peak:

```
F:\ZRespaldo\Escuela\4Semestre\PP\Practica1\Practica1_03.exe

ID  Usuario         Documento          Total_Pag   Restantes_pag   Copias  Prioridad   Estado
1   BOLSA           TAREA.pdf          5           10              2       Normal      En Cola
------- MENU -------
---- INVENTARIO ----
1- Agregar
2- Peak
3- Dequeue
4- Listar
5- Salir

Ingresa tu opcion:
```

Dequeue:

```
F:\ZRespaldo\Escuela\4Semestre\PP\Practica1\Practica1_03.exe

Imprmiendo la siguiente tarea: TAREA.pdf

Estado: Imprimiendo
----Copias faltantes: 1----
Hojas totales restantes: 5

Hojas faltantes de esta copia: 5
. . . . . .
Hojas faltantes de esta copia: 4
. . . . . .
Hojas faltantes de esta copia: 3
. . . . . .
Hojas faltantes de esta copia: 2
. . . . . .
Hojas faltantes de esta copia: 1
. . . . . .
```

Listar:

```
F:\ZRespaldo\Escuela\4Semestre\PP\Practica1\

La lista esta vacia

------- MENU -------
---- INVENTARIO ----
1- Agregar
2- Peak
3- Dequeue
4- Listar
5- Salir

Ingresa tu opcion:
```

Conclusión

El desarrollo de este proyecto permitió aplicar conceptos fundamentales de estructuras de datos, particularmente el uso de colas dinámicas mediante listas enlazadas.

Además, se reforzaron conocimientos sobre manejo de memoria dinámica, uso de punteros, alcance de variables y organización del código en funciones.

La simulación desarrollada representa de forma sencilla el funcionamiento de un sistema de cola de

impresión, demostrando cómo las estructuras de datos pueden aplicarse a problemas del mundo real.

Practica 1

3516e25de9d904a3048763173fbdd7b0421366f4